The density-fitted NEVPT2 energy: SC and PC, on CASCI and CASSCF orbitals

`nevpt2-gpu-demo`, `docs/energy.tex`

Abstract

`nevpt2_df_demo` computes the strongly-contracted (SC) and, with `--pc`, the partially-contracted (PC) NEVPT2 correlation energy of a density-fitted reference, and checks every class and the total against a committed golden file to $10^{-7}\,E_h$. The golden may be built on RHF orbitals (a CASCI reference) or on converged CASSCF orbitals. This document covers the four combinations $\{\mathrm{SC}, \mathrm{PC}\} \times \{\mathrm{CASCI}, \mathrm{CASSCF}\}$. For each one it gives the equations the code evaluates and pseudocode for the path through the tree. The pipeline in prose is `docs/implementation.md`, and the PC derivation and its measured behaviour are `docs/pc-nevpt2.md`. Like every file in `docs/`, this one states what the code *does*, and it carries no measured numbers.

Contents

1 The four paths, and where they actually differ **2**

2 Notation and conventions **2**

3 What a density-fitted golden carries **3**
3.1 Density fitting . . . 3
3.2 Canonicalisation and the one-electron inputs . . . 4
3.3 The reference answers . . . 4

4 Shared stage A: external integrals from B, slab by slab **4**

5 Shared stage B: $\Gamma^{(3)}$, F^{ca}, F^{ac} on the device **6**

6 The strongly-contracted energy **7**
6.1 The formula every class shares . . . 7
6.2 The eight classes . . . 7

7 The partially-contracted energy **9**
7.1 PC is SC without the final contraction . . . 9
7.2 One solve per class . . . 9
7.3 The eight classes . . . 10

8 Path 1: SC-NEVPT2 on a CASCI reference **11**

9 Path 2: PC-NEVPT2 on a CASCI reference **12**

10 Path 3: SC-NEVPT2 on a CASSCF reference **13**

11 Path 4: PC-NEVPT2 on a CASSCF reference **13**

12 Scope and limits **14**

A The RDM-only intermediates **14**
A.1 Hole-side metrics and the $\bar{\Gamma}$, R families 14
A.2 A^{16} (Sr), reading F^{ca} and F^{ac} 15
A.3 A^{22} (Si), reading F^{ca} and F^{ac} 15
A.4 The small blocks: A^{3} (Sijr), K^{27} (Srsi) 15
A.5 A^{7} (Srs), A^{9} (Sij) 15
A.6 A^{12}, A^{13} (Sir) 16
A.7 SC only: the one-body Dyall blocks A^{17}, A^{19} (Sr), A^{23}, A^{25} (Si) 16

1 The four paths, and where they actually differ

	CASCI orbitals (RHF)	CASSCF orbitals
SC	§8	§10
PC (`--pc`)	§9	§11

Table 1: The four paths, and the section that walks through each.

The two axes of Table 1 split the work at different places:

- **CASCI vs. CASSCF is decided offline, in the golden file.** Every golden is a deterministic `mcscf.CASCI` (density-fitted: `DFCASCI`). A CASSCF golden is the same CASCI, run on converged CASSCF orbitals that `generate_golden.py --casscf --mo-out` optimised once and committed as a `.mogold` sidecar, and that `--mo-from` reads back. The canonicalisation, the RDMs, the integral blocks and the reference answers all come downstream of that solve. The only trace in the file is the header field `reference_kind`. The demo prints it and otherwise ignores it. **At run time the CASCI and CASSCF paths run the same code.** (Beyond the energy, the field matters in one place: `requireLagrangianReference` refuses a `casscf` golden, because the Z-vector chain is derived for CASCI on RHF orbitals. The source-term checks pass `requireSourceTermReference`, which accepts both kinds. Neither is part of this document.)
- **SC vs. PC is decided at run time, by `--pc`.** Both share the input stage, the density-fitted slab builder (§4), the RDM build (§5) and the RDM-only intermediates (Appendix A). They fork at the class energies: `energiesDevice` (§6) or `pcEnergiesDevice` (§7).

So the four paths are two run-time pipelines, each fed by two kinds of golden. Sections 8–11 still give each path its own end-to-end pseudocode, offline step included, so that each can be read alone.

2 Notation and conventions

Orbital spaces. i, j label core (doubly occupied) orbitals, r, s virtual orbitals. Active orbitals are labelled by any of $p, q, a, b, c, t, u, k, l, m$. The counts are n_c, n_a (also written n) and n_v, and n_L is the number of auxiliary functions.

Integrals. $(pq|rs)$ is a chemists'-notation two-electron integral over the canonicalised MOs (§3). The active block that the RDM build and the intermediates read is stored in *physicists'* order,

$$v_{tuvw} \equiv \texttt{h2e}[t,u,v,w] = (tv|uw), \tag{1}$$

and its chemists'-order transpose is $e_{tuvw} = (tu|vw)$ (`transpose(h2e, {0,2,1,3})` in `src/rdm/rdm_build.cpp`). h is the active-active block of the effective one-electron operator `h1eff` (§3).

Density matrices. With $E_{pq} = \sum_\sigma a^\dagger_{p\sigma} a_{q\sigma}$ and the CAS reference $|\Psi_0\rangle$,

$$\Gamma^{(1)}_{pq} = \langle E_{pq}\rangle, \qquad \Gamma^{(2)}_{pqrs} = \langle E_{pq}E_{rs}\rangle, \qquad \Gamma^{(3)}_{pqrstu} = \langle E_{pq}E_{rs}E_{tu}\rangle, \tag{2}$$

which is the convention of PySCF's `fci.rdm.make_dm123` and of the golden's `dm1`/`dm2` (`docs/architecture.md`, the `rdm_host_dm` row). F^{ca} and F^{ac} are the two integral-contracted 4-RDMs (`f3ca`, `f3ac`), PySCF's `NEVPTkern_sf` quantities, built on the device in §5. The full 4-RDM is never formed.

Orbital energies. ε_i, ε_r are the diagonal of the generalised Fock matrix in the canonical core and virtual bases (§3), read from the golden as `e_core`, `e_virt`.

Summation. In the class formulas of §6 and §7, *repeated active indices are summed* and external indices are free unless a sum says otherwise. The intermediates in Appendix A write every sum explicitly.

Tolerances (constants of the code). `NUMERICAL_ZERO` $= 10^{-14}$ (the SC empty-perturber guard), `PC_TAU` $= 10^{-13}$ (the PC metric cut), `PC_MIN_GAP` $= 10^{3}$ (the PC gap refusal), and $10^{-7}\,E_h$ (the golden comparison), all in `src/energy/energy_finish.cppm` and the demo's `main`.

3 What a density-fitted golden carries

The demo's inputs are fixed offline by `reference_data/generate_golden.py --df`, the only file that defines the state. No PySCF runs at build or test time. This section states what the file contains, because each of the four paths begins here.

3.1 Density fitting

The whole calculation is fitted with one auxiliary basis (`--auxbasis`, default `<basis>-jkfit`): the RHF, the CASCI (or CASSCF), the canonicalisation, `h1eff` and every NEVPT2 integral block. With $J_{PQ} = (P|Q)$ the Coulomb metric and M any factor with $M^{\mathsf{T}}M = J^{-1}$ (PySCF's `_cderi`),

$$B^L_{pq} = \sum_{\mu\nu} C_{\mu p} C_{\nu q} \sum_P M_{LP}\,(P|\mu\nu), \qquad (pq|rs) \approx \sum_{L=1}^{n_L} B^L_{pq}\,B^L_{rs} \tag{3}$$

(the symmetric-metric convention: no metric between the two factors). The file carries four MO blocks of B,

$$B_{aa}\,(n_L,n_a,n_a), \quad B_{ca}\,(n_L,n_c,n_a), \quad B_{va}\,(n_L,n_v,n_a), \quad B_{cv}\,(n_L,n_c,n_v), \tag{4}$$

and **no virtual–virtual block**. No SC- or PC-NEVPT2 class has two virtual indices in one charge distribution, so B_{vv}, the largest block of all, is never needed.

3.2 Canonicalisation and the one-electron inputs

After the CAS solve the generator calls `NEVPT.canonicalize(cas_natorb=True)`, which is exactly what `pyscf.mrpt.NEVPT.kernel` does. It forms the generalised Fock matrix

$$f_{pq} = h^{\mathrm{core}}_{pq} + \sum_i \big[2(pq|ii) - (pi|iq)\big] + \sum_{tu} \Gamma^{(1)}_{tu} \big[(pq|tu) - \tfrac{1}{2}(pt|uq)\big], \tag{5}$$

diagonalises its core–core and virtual–virtual blocks (ε_i, ε_r are the eigenvalues), and rotates the active space to natural orbitals, transforming the CI vector with it. The effective one-electron operator that the active space sees is the core Fock operator,

$$h^{\mathrm{eff}}_{pq} = h^{\mathrm{core}}_{pq} + \sum_i \big[2(pq|ii) - (pi|iq)\big], \tag{6}$$

and the golden stores four slices of it (`_integral_blocks`):

$$h_{pq} = h^{\mathrm{eff}}_{pq}, \quad h'^{\mathrm{Sr}}_{rp} = h^{\mathrm{eff}}_{rp} - \sum_b (rb|bp), \quad h'^{\mathrm{Si}}_{pi} = h^{\mathrm{eff}}_{pi}, \quad h'^{\mathrm{Sir}}_{ri} = h^{\mathrm{eff}}_{ri}. \tag{7}$$

These, ε, $\Gamma^{(1)}$ and $\Gamma^{(2)}$ are **read from the file**. They belong to the CAS calculation and are not a fitting target, so the DF demo does not rebuild them. $\Gamma^{(3)}$, F^{ca} and F^{ac} are deliberately *not* in the file: building them is the device's job (§5).

3.3 The reference answers

The SC reference is PySCF's DF-NEVPT2 (`pyscf.mrpt.dfnevpt2`) on the same state: per-class norms and energies and the total. The PC reference is block2's `icnevpt2_full` with the same fitted integrals injected (`docs/pc-nevpt2.md`, "The block2 reference"), stored as `pc_class_energies` and `e_pc_total`. A golden whose PC metric is not gapped is written `--no-pc` and carries no PC fields. N2 CAS(12,12) is one (`docs/implementation.md`, §4.3).

4 Shared stage A: external integrals from B, slab by slab

The class energies never read a whole four-index external block. Each class walks one external index in batches of `--batch` (default 8; 0 means one batch), and asks its `IntegralSource` for the slab covering that batch. `DfIntegralSource` (`src/df_integrals/`) builds every slab the same way: one DGEMM and at most one permutation.

$$C_{xyzw} = \sum_L X^L_{xy}\, Y^L_{zw}, \qquad x \in [b_0, b_1) \text{ (the batch)}, \tag{8}$$

$$\mathrm{slab}_{\sigma(xyzw)} = C_{xyzw} \tag{9}$$

Equation (8) is a single `wwrblasDgemm` with $M = n_z n_w$, $N = (b_1 - b_0)\, n_y$, $K = n_L$, whose B operand is a contiguous run of X's rows. Equation (9) is the generic einsum kernel's transpose into the physicist layout the class subscripts expect. The batched index is always the first index of X's pair, which is why B_{cv} is also held transposed as B_{vc}. Table 2 is the code's own table (`DfIntegralSource::slab`).

Only the slab-sized C and its permuted copy exist at any one time. The caller drops each slab at the end of its batch iteration, so the external memory high-water mark is set by `--batch` and not by the size of the blocks (`docs/reference-data.md`, "Density fitting", has what that measured).

block	slab element	X	Y	C	axes σ
V^{Sr}_{rabc}	$(rb\|ac)$	B_{va}	B_{aa}	$[r,b,a,c]$	0 2 1 3
V^{Si}_{tuiv}	$(it\|uv)$	B_{ca}	B_{aa}	$[i,t,u,v]$	1 2 0 3
g_{iajb} (Sijrs)	$(ia\|jb)$	B_{cv}	B_{cv}	$[i,a,j,b]$	identity
V^{Sijr}_{rpji}	$(rj\|ip)$	B_{vc}	B_{ca}	$[r,j,i,p]$	0 3 1 2
V^{Srsi}_{rsit}	$(ri\|st)$	B_{vc}	B_{va}	$[r,i,s,t]$	0 2 1 3
$V^{\mathrm{SrsiT}}_{srit}$	$(rt\|si)$	B_{va}	B_{vc}	$[r,t,s,i]$	2 0 3 1
V^{Srs}_{rstu}	$(rt\|su)$	B_{va}	B_{va}	$[r,t,s,u]$	0 2 1 3
V^{Sij}_{tuij}	$(it\|ju)$	B_{ca}	B_{ca}	$[i,t,j,u]$	1 3 0 2
V^{1}_{rtiu} (Sir)	$(ri\|tu)$	B_{vc}	B_{aa}	$[r,i,t,u]$	0 2 1 3
V^{2}_{ruti} (Sir)	$(rt\|iu)$	B_{va}	B_{ca}	$[r,t,i,u]$	0 3 1 2
v_{tuvw} (active)	$(tv\|uw)$	B_{aa}	B_{aa}	$[t,v,u,w]$	0 2 1 3

Table 2: Each external block as one DGEMM over B plus a permutation (slab axis k is C axis σ_k, NumPy's convention). The first index is the batched one, except for Si and Sij (axis 2) and SrsiT (axis 1, the Srsi block batched along its second virtual, because Srsi contracts $V_{rs\cdots}$ against $V_{sr\cdots}$). The last row, the active block, is built whole, once, before the RDM build.

Algorithm 1 `DfIntegralSource::slab` and `activeH2e`

Input: block b, batch $[b_0, b_1)$; device-resident $B_{aa}, B_{ca}, B_{va}, B_{cv}$ and $B_{vc} = B_{cv}^{\mathsf{T}_{12}}$

$(X, Y, \sigma) \leftarrow$ the row of Table 2 for b

$C \leftarrow \mathbf{0}_{(b_1-b_0)\times n_y\times n_z\times n_w}$ ▷ from the pool, on the one stream

$C \leftarrow \textsc{Dgemm}\big(Y^{\mathsf{T}}_{(L,zw)},\ X_{(L,\,b_0:b_1\,y)}\big)$ ▷ Eq. (8)

if σ is the identity **then return** C

$S \leftarrow \textsc{Transpose}(C, \sigma)$ ▷ Eq. (9)

return S ▷ C is freed stream-ordered, behind the transpose

5 Shared stage B: $\Gamma^{(3)}$, F^{ca}, F^{ac} on the device

`nevpt2.rdm_build` is shared by both demos, and the only integrals it reads are the active block v. The DF demo builds v from B_{aa} (last row of Table 2), downloads it once, and hands it to the same `buildRdmsDevice` the integral-direct demo calls. The determinant axis is split into `--tiles` tiles. Per tile, with K a determinant of the tile,

$$R_{tu,K} = \left(E_{tu}|\Psi_0\right)_K, \qquad L_{pqrs,K} = \left(E_{pq}E_{rs}|\Psi_0\right)_K, \tag{10}$$

$$W^{\mathrm{ca}}_{af,K} = \sum_{pqx} e_{axqp}\, L_{pqxf,K}, \qquad W^{\mathrm{ac}}_{af,K} = \sum_{pqx} e_{axqp}\, L_{pqfx,K}, \tag{11}$$

and the three digests accumulate over tiles into device-resident totals:

$$\Gamma^{(3)}_{[pqrs],[tu]} \mathrel{+}= \sum_K L_{pqrs,K}\, R_{tu,K}, \tag{12}$$

$$F^{\mathrm{ca}}_{[w],f,a} \mathrel{+}= \sum_K W^{\mathrm{ca}}_{af,K}\, L_{w,K}, \qquad F^{\mathrm{ac}}_{[w],a,f} \mathrel{+}= \sum_K W^{\mathrm{ac}}_{af,K}\, L_{w,K}, \tag{13}$$

where $[w]$ is a four-index row of L. The products in Eqs. (10)–(13) use the string conventions of PySCF's `fci.rdm` kernels, which fix the index order that the golden's $\Gamma^{(1)}, \Gamma^{(2)}$ and the energy einsums expect. The digest's target row is L's own row, so no permuted copy of L is ever made. After the tile loop, two corrections run once, on the fully summed tensors, for each order $o \in \{\mathrm{ca}, \mathrm{ac}\}$:

$$f^{\mathrm{ca}}_{ijkl} = \sum_{ABC} e_{lCBA}\, \Gamma^{(3)}_{ABCkji}, \qquad f^{\mathrm{ac}}_{ijkl} = \sum_{CEF} e_{kCEF}\, \Gamma^{(3)}_{ijClEF}, \tag{14}$$

and the "wedge", for every $j < i$ and all free indices:

$$F^o_{jbicde} \leftarrow F^o_{icjbde}, \qquad F^o_{jiicde} \mathrel{+}= f^o_{jcde}, \qquad F^o_{jbijde} \mathrel{-}= f^o_{ibde}. \tag{15}$$

Algorithm 2 `buildRdmsDevice` (the default emitted digest; the GEMM digests differ only in where the per-tile add happens)

Input: CI vector c, (n_a, n_α, n_β), active v, tile count T

1: link tables $\leftarrow$ KnowlesHandy(n_a, n_α, n_β) ▷ host
2: $e \leftarrow$ Transpose$(v, (0,2,1,3))$; upload c, e, link tables
3: $\Gamma^{(3)}, F^{\mathrm{ca}}, F^{\mathrm{ac}} \leftarrow \mathbf{0}_{n^6}$ ▷ device accumulators
4: **for** each determinant tile $\mathcal{T}$ of T **do**
5: $\quad (R, L) \leftarrow$ Produce$(c, \mathcal{T})$ ▷ Eq. (10), one thread per K
6: $\quad W^{\mathrm{ca}}, W^{\mathrm{ac}} \leftarrow$ Consume(e, L) ▷ Eq. (11), strided-batched DGEMM
7: $\quad \Gamma^{(3)} \mathrel{+}= L\, R^{\mathsf{T}}$; $\quad F^{\mathrm{ca}} \mathrel{+}= \cdots$; $\quad F^{\mathrm{ac}} \mathrel{+}= \cdots$ ▷ Eqs. (12)–(13)
8: **for** $o \in \{\mathrm{ca}, \mathrm{ac}\}$ **do**
9: $\quad f^o \leftarrow$ Fdm2$(e, \Gamma^{(3)})$; $\quad F^o \leftarrow$ Wedge(F^o, f^o) ▷ Eqs. (14)–(15)
10: **return** $\Gamma^{(3)}, F^{\mathrm{ca}}, F^{\mathrm{ac}}$ ▷ device-resident, never downloaded

The digests can run four ways: emitted kernels, native fp64 BLAS (`--blas-digest`), cuBLAS fixed-point emulation (`--cublas`, CUDA only) and int8 Ozaki (`--ozaki`). All four compute Eqs. (12)–(13). Which is the default, and why, is `docs/performance.md`, "Native-fp64 BLAS digest". `--tiles` is a memory lever, and the floors are in `docs/performance.md`, "Tile floors".

6 The strongly-contracted energy

6.1 The formula every class shares

For a perturber class κ (one of Sr, Si, Sijrs, Sijr, Srsi, Srs, Sij, Sir, the order PySCF and the golden use) and an external tuple t of that class, SC-NEVPT2 builds one perturber $|\Psi_t^\kappa\rangle = P_t^\kappa V|\Psi_0\rangle$, the projection of the perturbation onto the internal space that t labels. With the Dyall Hamiltonian H_D and $E_0 = \langle\Psi_0|H_D|\Psi_0\rangle$,

$$N_t = \langle\Psi_t^\kappa|\Psi_t^\kappa\rangle, \qquad E_t^\kappa - E_0 = \Delta_t + \frac{H_t}{N_t}, \qquad E_\kappa^{(2)} = -\sum_{t:\,|N_t|>\texttt{NUMERICAL_ZERO}} \frac{N_t}{\Delta_t + H_t/N_t}, \tag{16}$$

where Δ_t is the tuple's orbital-energy sum (Table 3) and H_t is the active part of $\langle\Psi_t^\kappa|(H_D - E_0)|\Psi_t^\kappa\rangle$. The total is $E^{(2)} = \sum_\kappa E_\kappa^{(2)}$. The reduction in Eq. (16) is `normToEnergy`, a verbatim transcription of `pyscf.mrpt.nevpt2._norm_to_energy`, vanishing-norm guard included. The class norm it returns is the plain sum $\sum_t N_t$, which is what the golden's `class_norms` holds.

Each N_t and H_t is a quadratic form in the slab's integrals, contracted against one RDM-only block per term: a metric on the N side and a Dyall block on the H side. Neither block depends on t. Every class therefore builds its blocks *once*, before its slab walk, and the walk does one einsum chain per slab. The blocks are listed in Appendix A. `make_a16` and `make_a22` are the only readers of F^{ca}, F^{ac}.

6.2 The eight classes

Each formula below is the einsum chain of `src/energy/energy_sc_classes.cppm`, with the code's subscripts renamed so that core indices are i, j, virtual indices are r, s, and every other index is active (summed). The slabs V are those of Table 2, and the one-electron rows those of Eq. (7).

Sr **(one virtual),** $\Delta_r = \varepsilon_r$**.**

$$N_r = V^{\mathrm{Sr}}_{rpqu}\,\Gamma^{(3)}_{upqbac}\,V^{\mathrm{Sr}}_{rabc} + 2\,V^{\mathrm{Sr}}_{rpqu}\,\Gamma^{(2)}_{upqa}\,h'^{\mathrm{Sr}}_{ra} + h'^{\mathrm{Sr}}_{rp}\,\Gamma^{(1)}_{pa}\,h'^{\mathrm{Sr}}_{ra}, \tag{17}$$

$$H_r = V^{\mathrm{Sr}}_{rpqu}\,A^{16}_{pqu,abc}\,V^{\mathrm{Sr}}_{rabc} + 2\,V^{\mathrm{Sr}}_{rpqu}\,A^{17}_{pqua}\,h'^{\mathrm{Sr}}_{ra} + h'^{\mathrm{Sr}}_{rp}\,A^{19}_{pa}\,h'^{\mathrm{Sr}}_{ra}. \tag{18}$$

Si **(one core),** $\Delta_i = -\varepsilon_i$**.** The metric is hole-side: $\Gamma^{\mathrm{h}(3)}$, $\Gamma^{\mathrm{h}(2)}$ and $\Gamma^{\mathrm{h}(1)} = 2\delta - \Gamma^{(1\,\mathsf{T})}$ (Eqs. (40)–(41)).

$$N_i = V^{\mathrm{Si}}_{qpiu}\,\Gamma^{\mathrm{h}(3)}_{upqbac}\,V^{\mathrm{Si}}_{baic} + 2\,V^{\mathrm{Si}}_{qpiu}\,\Gamma^{\mathrm{h}(2)}_{upqa}\,h'^{\mathrm{Si}}_{ai} + h'^{\mathrm{Si}}_{pi}\,\Gamma^{\mathrm{h}(1)}_{pa}\,h'^{\mathrm{Si}}_{ai}, \tag{19}$$

$$H_i = V^{\mathrm{Si}}_{qpiu}\,A^{22}_{pqu,abc}\,V^{\mathrm{Si}}_{baic} + 2\,V^{\mathrm{Si}}_{qpiu}\,A^{23}_{pqua}\,h'^{\mathrm{Si}}_{ai} + h'^{\mathrm{Si}}_{pi}\,A^{25}_{pa}\,h'^{\mathrm{Si}}_{ai}. \tag{20}$$

Sijrs **(two core, two virtual).** It has no active index and no RDM, so it is MP2-like. It runs as a dedicated reduction kernel (`energy_sijrs`) over (i, a, j, b) slabs of $g_{iajb} = (ia|jb)$, with a, b virtual here:

$$N = \sum_{iajb} g_{iajb}\,\theta_{iajb}, \qquad E^{(2)}_{\mathrm{Sijrs}} = \sum_{iajb} \frac{g_{iajb}\,\theta_{iajb}}{\varepsilon_i + \varepsilon_j - \varepsilon_a - \varepsilon_b}, \qquad \theta_{iajb} = 2g_{iajb} - g_{ibja}. \tag{21}$$

Each thread accumulates both sums in a grid-stride loop. A warp-shuffle block reduction then folds each sum into one `atomicAdd` per block.

Sijr **(two core, one virtual).** With $G = \bar{\Gamma}^{(1)} = 2\delta - \Gamma^{(1\,\mathsf{T})}$ on the N side and $G = A^3$ on the H side,

$$\tilde{X}_{r,ji}[G] = 2\,V^{\mathrm{Sijr}}_{rpji}\,V^{\mathrm{Sijr}}_{raji}\,G_{pa} - V^{\mathrm{Sijr}}_{rpji}\,V^{\mathrm{Sijr}}_{raij}\,G_{pa}. \tag{22}$$

The host symmetrises in (i,j), halves the diagonal and packs $j \le i$:

$$X_{r,ji} = (\tilde{X}_{r,ji} + \tilde{X}_{r,ij})(1 - \tfrac{1}{2}\delta_{ij}), \quad N_{rji} = X_{r,ji}[\bar{\Gamma}^{(1)}], \quad H_{rji} = X_{r,ji}[A^3], \quad \Delta_{rji} = \varepsilon_r - \varepsilon_j - \varepsilon_i. \tag{23}$$

Srsi **(two virtual, one core).** With $G = \Gamma^{(1)}$ (N) and $G = K^{27}$ (H), accumulated into a whole (n_v, n_v, n_c) table, batch by batch,

$$\tilde{X}_{rs,i}[G] = 2\,V^{\mathrm{Srsi}}_{rsip}\,V^{\mathrm{Srsi}}_{rsia}\,G_{pa} - V^{\mathrm{Srsi}}_{rsip}\,V^{\mathrm{SrsiT}}_{sria}\,G_{pa}. \tag{24}$$

After the walk, the host symmetrises in (r,s), halves the diagonal and packs $r \le s$, with $\Delta_{rsi} = \varepsilon_r + \varepsilon_s - \varepsilon_i$. The table is whole because (r,s) and (s,r) can fall in different batches.

Srs **(two virtual),** $\Delta_{rs} = \varepsilon_r + \varepsilon_s$**, over ordered** (r,s)**.**

$$N_{rs} = \tfrac{1}{2}\,V^{\mathrm{Srs}}_{rsqp}\,V^{\mathrm{Srs}}_{rsba}\,R^{(2)}_{pqba}, \qquad H_{rs} = \tfrac{1}{2}\,V^{\mathrm{Srs}}_{rsqp}\,V^{\mathrm{Srs}}_{rsba}\,A^{7}_{pqab}. \tag{25}$$

Sij **(two core),** $\Delta_{ij} = -(\varepsilon_i + \varepsilon_j)$**, over ordered** (i,j)**.**

$$N_{ij} = \tfrac{1}{2}\,V^{\mathrm{Sij}}_{qpij}\,V^{\mathrm{Sij}}_{baij}\,\bar{\Gamma}^{(2)}_{pqab}, \qquad H_{ij} = \tfrac{1}{2}\,V^{\mathrm{Sij}}_{qpij}\,V^{\mathrm{Sij}}_{baij}\,A^{9}_{pqab}. \tag{26}$$

Sir **(one core, one virtual),** $\Delta_{ir} = \varepsilon_r - \varepsilon_i$**.** Two slabs, V^1 and V^2, and the row $h'_{ri} \equiv h'^{\mathrm{Sir}}_{ri}$:

$$\begin{aligned} N_{ir} = {} & 2\,V^1_{rpiq}V^1_{raib}\Gamma^{(2)}_{qpab} - V^1_{rpiq}V^2_{rabi}\Gamma^{(2)}_{qpab} - V^2_{rpqi}V^1_{raib}\Gamma^{(2)}_{qpab} + 2\,V^2_{raqi}V^2_{rabi}\Gamma^{(1)}_{qb} \\ & - V^2_{rpqi}V^2_{rabi}\Gamma^{(2)}_{qbap} + V^2_{rpqi}\Big(\textstyle\sum_a V^2_{raai}\Big)\Gamma^{(1)}_{qp} + 4\,V^1_{rpiq}h'_{ri}\Gamma^{(1)}_{qp} - 2\,V^2_{rpqi}h'_{ri}\Gamma^{(1)}_{qp} + 2\,h'^2_{ri}, \end{aligned} \tag{27}$$

$$H_{ir} = 2\,V^1_{rpiq}V^1_{raib}A^{12}_{pqab} - V^1_{rpiq}V^2_{rabi}A^{12}_{pqab} - V^2_{rpqi}V^1_{raib}A^{12}_{pqab} + V^2_{rpqi}V^2_{rabi}A^{13}_{pqab}. \tag{28}$$

class	tuple t	metric (norm)	Dyall block (H)	Δ_t
Sr	r	$\Gamma^{(3)}, \Gamma^{(2)}, \Gamma^{(1)}$	A^{16}, A^{17}, A^{19}	ε_r
Si	i	$\Gamma^{\mathrm{h}(3)}, \Gamma^{\mathrm{h}(2)}, \Gamma^{\mathrm{h}(1)}$	A^{22}, A^{23}, A^{25}	$-\varepsilon_i$
Sijrs	(i,j,r,s)	—	—	$\varepsilon_r + \varepsilon_s - \varepsilon_i - \varepsilon_j$
Sijr	$(r, j \le i)$	$\bar{\Gamma}^{(1)}$	A^3	$\varepsilon_r - \varepsilon_i - \varepsilon_j$
Srsi	$(r \le s, i)$	$\Gamma^{(1)}$	K^{27}	$\varepsilon_r + \varepsilon_s - \varepsilon_i$
Srs	(r,s)	$R^{(2)}$	A^7	$\varepsilon_r + \varepsilon_s$
Sij	(i,j)	$\bar{\Gamma}^{(2)}$	A^9	$-\varepsilon_i - \varepsilon_j$
Sir	(i,r)	$\Gamma^{(2)}, \Gamma^{(1)}$	A^{12}, A^{13}	$\varepsilon_r - \varepsilon_i$

Table 3: The SC classes at a glance.

Algorithm 3 `energiesDevice` over a `DfIntegralSource`

Input: active ints $(h, v, h'^{\mathrm{Sr}}, h'^{\mathrm{Si}}, h'^{\mathrm{Sir}}, \varepsilon)$; $\Gamma^{(1)}, \Gamma^{(2)}$ (uploaded); $\Gamma^{(3)}, F^{\mathrm{ca}}, F^{\mathrm{ac}}$ (device); source $\mathcal{S}$; batch β

Output: $(N_\kappa, E_\kappa^{(2)})$ for the eight classes, $E^{(2)}$

for $\kappa \in (\mathrm{Sr}, \mathrm{Si}, \mathrm{Sijrs}, \mathrm{Sijr}, \mathrm{Srsi}, \mathrm{Srs}, \mathrm{Sij}, \mathrm{Sir})$ **do**
  if $\kappa = \mathrm{Sijrs}$ **then**
    for each core batch $[b_0, b_1)$ **do**
      $g \leftarrow \mathcal{S}.\textsc{slab}(\mathrm{Sijrs}, b_0, b_1)$; (N, E) += EnergySijrs(g, ε) ▷ Eq. (21)
    continue
  build the class's metric and Dyall blocks ▷ App. A; once per class
  for each batch $[b_0, b_1)$ of the class's external index **do** ▷ `forBatches`
    $V \leftarrow \mathcal{S}.\textsc{slab}(\kappa, b_0, b_1)$ ▷ Alg. 1; Srsi: also SrsiT; Sir: V^1, V^2
    $N_t, H_t \leftarrow$ the class's einsum chains on V ▷ §6.2
    download N_t, H_t ▷ synchronises the stream; no sync before it
    Sijr: symmetrise and pack $j \le i$ on the host
    $(N_\kappa, E_\kappa^{(2)})$ += NormToEnergy(N_t, H_t, Δ_t) ▷ Eq. (16)
  Srsi: symmetrise and pack $r \le s$ over the whole table, then NormToEnergy
return $N_\kappa, E_\kappa^{(2)}$, $E^{(2)} = \sum_\kappa E_\kappa^{(2)}$

7 The partially-contracted energy

7.1 PC is SC without the final contraction

SC collapses each class to two scalars per tuple. PC keeps the whole span of the class's internal functions. With S the class metric, K its Dyall block (the same blocks SC contracts, as $d \times d$ matrices over the class's internal basis), x_t the tuple's slab as a vector over that basis, S_1 the metric columns through which the one-body part of $V|\Psi_0\rangle$ projects, and h_t the tuple's one-electron row,

$$E_t^{\mathrm{PC}} = -c\, b_t^{\mathsf{T}} \left(K + \Delta_t S\right)^{+} b_t, \qquad b_t = S\, x_t + S_1\, h_t. \tag{29}$$

Here c is the class's coupling factor: 1 for Sr, Si and Sir, $\frac{1}{2}$ over ordered pairs for Srs and Sij, and the two-coupling form of Eq. (36) for Sijr and Srsi (Table 4). In one dimension Eq. (29) reduces exactly to Eq. (16), and $K + \Delta_t S$ is block2's H_D. **PC needs no new RDM intermediate**: it calls the same `make_a16`, `make_a22`, `make_a7`, `make_hdm1`, `make_a3`, `make_k27`, `make_a9`, `make_a12` and `make_a13` as SC. It does not need SC's one-body Dyall blocks $A^{17}, A^{19}, A^{23}, A^{25}$, because the one-body part enters only through $S_1 h_t$. Sijrs has no active index, so there PC equals SC, and `pcEnergiesDevice` calls `energy_Sijrs` itself.

7.2 One solve per class

S and K are the same for every tuple, and only Δ_t and b_t vary. So `solveClass` diagonalises each class once:

$$\bar{S} = \tfrac{1}{2}(S + S^{\mathsf{T}}) = V\,\mathrm{diag}(s)\,V^{\mathsf{T}}, \qquad \bar{K} = \tfrac{1}{2}(K + K^{\mathsf{T}}), \tag{30}$$

$$\mathcal{K} = \{k : s_k > \texttt{PC_TAU} \cdot s_{\max}\}, \qquad X = V_{\mathcal{K}}\,\mathrm{diag}(s_{\mathcal{K}})^{-1/2}, \tag{31}$$

$$X^{\mathsf{T}} \bar{K} X = W\,\mathrm{diag}(\lambda)\,W^{\mathsf{T}}, \qquad U = XW, \tag{32}$$

$$T = \bar{S}\,U, \qquad t_1 = S_1^{\mathsf{T}} U. \tag{33}$$

On the kept range, $(K+\Delta_t S)^+ = U\,\mathrm{diag}\big(1/(\lambda_k+\Delta_t)\big)\,U^{\mathsf{T}}$. Each slab is then a single einsum straight into the eigenbasis, and the per-tuple energy finishes on the host:

$$y_t = T^{\mathsf{T}} x_t + t_1^{\mathsf{T}} h_t, \qquad E_t^{\mathrm{PC}} = -c \sum_{k\in\mathcal{K}} \frac{y_{tk}^2}{\lambda_k+\Delta_t}. \tag{34}$$

K is symmetrised because a quadratic form defines only its symmetric part, and $\langle\Phi|[H_D,\Phi]\rangle$ is Hermitian only for an exact eigenstate. The two eigendecompositions are `wwrsolverDnDsyevd`, and the $d\times d$ products are `wwrblasDgemm`.

Refusals. A class gets *no* energy, and is reported refused, when any of the following holds:

- $\bar{S}$ has no positive eigenvalue;
- the gap is too small (`pcRankCut`):

$$\frac{\min_{k\in\mathcal{K}} s_k/s_{\max}}{\max_{k\notin\mathcal{K}} |s_k|/s_{\max}} < \texttt{PC_MIN_GAP} \quad \text{(denominator } \texttt{PC_TAU} \text{ when nothing is dropped);} \tag{35}$$

- some denominator $\lambda_k+\Delta_t \le 0$ (`checkDenominator`).

A `syevd` that runs but returns `devInfo` $\neq 0$ is a numerical error, which `main` reports. The gap is checked rather than assumed because S is singular by construction: Sr and Si drop exact null modes at every active space. That is also why a generalised eigensolver, which Cholesky-factors S, is not used (`docs/pc-nevpt2.md`, "The solve: once per class, not once per tuple").

7.3 The eight classes

class	basis, d	S	K	S_1 (row)	per-tuple finish
Sr	(p,q,u), n^3	$\Gamma^{(3)}_{upqbac}$	A^{16}	$\Gamma^{(2)}_{upqa}$ (h'^{Sr}_{ra})	$-\sum_k y_{rk}^2/(\lambda_k+\varepsilon_r)$
Si	(p,q,u), n^3	$\Gamma^{\mathrm{h}(3)}_{upqbac}$	A^{22}	$\Gamma^{\mathrm{h}(2)}_{upqa}$ (h'^{Si}_{ai})	$-\sum_k y_{ik}^2/(\lambda_k-\varepsilon_i)$
Sijr	p, n	$\bar{\Gamma}^{(1)}$	A^3	—	Eq. (36)
Srsi	p, n	$\Gamma^{(1)}$	K^{27}	—	Eq. (36)
Srs	(p,q), n^2	$R^{(2)}_{pqba}$	A^7	—	$-\frac{1}{2}\sum_{rs}\sum_k y_{rsk}^2/(\lambda_k+\varepsilon_r+\varepsilon_s)$
Sij	(p,q), n^2	$\bar{\Gamma}^{(2)}$	A^9	—	$-\frac{1}{2}\sum_{ij}\sum_k y_{ijk}^2/(\lambda_k-\varepsilon_i-\varepsilon_j)$
Sir	$(p,q)\oplus(p,q)$, $2n^2$	Eq. (37)	Eq. (38)	$(2\Gamma^{(1)}_{qp};\,-\Gamma^{(1)}_{qp})$ (h'^{Sir}_{ri})	$-\sum_k y_{irk}^2/(\lambda_k+\varepsilon_r-\varepsilon_i)$
Sijrs	—	—	—	—	SC's, Eq. (21)

Table 4: The PC classes (`src/energy/energy_pc_classes.cppm`). The pairs in Srs and Sij are ordered, with the factor $\frac{1}{2}$, as SC walks them. Neither class has an explicit $\pm$ split.

The slab einsums that produce y_t are

$$\begin{aligned}
\text{Sr}: y_{rk} &= V^{\mathrm{Sr}}_{rpqu} T_{pqu,k} + h'^{\mathrm{Sr}}_{ra} t_{1,ak}, & \text{Si}: y_{ik} &= V^{\mathrm{Si}}_{qpiu} T_{pqu,k} + h'^{\mathrm{Si}}_{ai} t_{1,ak},\\
\text{Sijr}: y_{rjik} &= V^{\mathrm{Sijr}}_{rpji} T_{pk}, & \text{Srsi}: y_{rsik} &= V^{\mathrm{Srsi}}_{rsip} T_{pk}, \quad y^{\mathsf{T}}_{rsik} = V^{\mathrm{SrsiT}}_{srip} T_{pk},\\
\text{Srs}: y_{rsk} &= V^{\mathrm{Srs}}_{rsqp} T_{pq,k}, & \text{Sij}: y_{ijk} &= V^{\mathrm{Sij}}_{qpij} T_{pq,k},\\
\text{Sir}: y_{irk} &= V^{1}_{rpiq} T^{(1)}_{pq,k} + V^{2}_{rpqi} T^{(2)}_{pq,k} + h'^{\mathrm{Sir}}_{ri} t_{1,k}.
\end{aligned}$$

Sijr, Srsi**: two spin couplings, no explicit ± split.** SC's "2 direct − exchange" coupling, summed over *ordered* external pairs:

$$E^{\mathrm{PC}}_{\mathrm{Sijr}} = -\sum_{r,j,i}\sum_{k} \frac{2y^2_{rjik} - y_{rjik}\,y_{rijk}}{\lambda_k + \varepsilon_r - \varepsilon_j - \varepsilon_i}, \qquad E^{\mathrm{PC}}_{\mathrm{Srsi}} = -\sum_{r,s,i}\sum_{k} \frac{2y^2_{rsik} - y_{rsik}\,y^{\mathsf{T}}_{rsik}}{\lambda_k + \varepsilon_r + \varepsilon_s - \varepsilon_i}. \tag{36}$$

Each Srsi batch holds both y and y^{T}, so unlike SC, PC needs no whole (n_v, n_v, n_c) table.

Sir**: the two couplings as two blocks.** The basis is two copies of (p, q), held as $(x, p, q;\ y, a, b)$ with $x, y \in \{1, 2\}$:

$$S = \begin{pmatrix} 2\Gamma^{(2)}_{qpab} & -\Gamma^{(2)}_{qpab} \\ -\Gamma^{(2)}_{qpab} & S^{22}_{pqab} \end{pmatrix}, \qquad S^{22}_{pqab} = 2\delta_{pa}\Gamma^{(1)}_{qb} - \Gamma^{(2)}_{qbap} + \delta_{ab}\Gamma^{(1)}_{qp}, \tag{37}$$

$$K = \begin{pmatrix} 2A^{12}_{pqab} & -A^{12}_{pqab} \\ -A^{12}_{pqab} & A^{13}_{pqab} \end{pmatrix}, \tag{38}$$

and T splits into its two n^2-row halves $T^{(1)}, T^{(2)}$.

Algorithm 4 `pcEnergiesDevice` over a `DfIntegralSource`

Input: as Algorithm 3
Output: per class: status $\in \{\textsc{Done}, \textsc{Refused}\}$, E^{PC}_{κ}, S's spectrum

for $\kappa \in$ (Sr, Si, Sijrs, Sijr, Srsi, Srs, Sij, Sir) **do**
  if $\kappa =$ Sijrs **then** $E^{\mathrm{PC}}_{\kappa} \leftarrow$ SC's `energy_Sijrs`; **continue**
  $S, K, S_1 \leftarrow$ Table 4, from App. A ▷ scoped: freed before the walk
  $\bar{S} \leftarrow \frac{1}{2}(S + S^{\mathsf{T}})$; $\bar{K} \leftarrow \frac{1}{2}(K + K^{\mathsf{T}})$
  $(V, s) \leftarrow \textsc{Syevd}(\bar{S})$ ▷ error if `devInfo` $\neq 0$
  $\mathcal{K}$, status $\leftarrow \textsc{PcRankCut}(s)$ ▷ Eqs. (31), (35)
  if status $=$ Refused **then** record why; **continue**
  $X \leftarrow V_{\mathcal{K}}\, s_{\mathcal{K}}^{-1/2}$; $(W, \lambda) \leftarrow \textsc{Syevd}(X^{\mathsf{T}}\bar{K}X)$; $U \leftarrow XW$
  $T \leftarrow \bar{S}U$; $t_1 \leftarrow S_1^{\mathsf{T}}U$ ▷ Eq. (33)
  $E \leftarrow 0$; $\delta_{\min} \leftarrow +\infty$
  for each batch $[b_0, b_1)$ **do**
    $V \leftarrow \mathcal{S}.\textsc{slab}(\kappa, b_0, b_1)$ ▷ Srsi: also SrsiT; Sir: V^1, V^2
    $y \leftarrow$ the class's slab einsum into the eigenbasis; download
    $E \mathrel{+}=$ the class's finish (Table 4); $\delta_{\min} \leftarrow \min(\delta_{\min}, \lambda_k + \Delta_t)$
  if $\delta_{\min} \le 0$ **then** status $\leftarrow$ Refused
  else $E^{\mathrm{PC}}_{\kappa} \leftarrow E$

8 Path 1: SC-NEVPT2 on a CASCI reference

Offline. The state is RHF → `DFCASCI` → NEVPT canonicalisation, all fitted with one auxiliary basis. The golden stores B (Eq. (4)), the one-electron slices (Eq. (7)), ε, $\Gamma^{(1)}$, $\Gamma^{(2)}$, the CI vector and PySCF's DF-NEVPT2 answer. Its header says `reference=casci`.

The equations are Eqs. (3), (8)–(15) and (16)–(21). The printed total is $E_{\mathrm{CASCI}} + E^{(2)}$. `--check-blocks` is a diagnostic. It also builds every external block whole from B and compares it with the four-index arrays that a `--df` golden carries as well, to 10^{-10}. The

Algorithm 5 Path 1, SC on CASCI orbitals

Offline (`generate_golden.py --df`; single-threaded, Davidson to 10^{-12}, point group detected)

1: $\phi \leftarrow$ DF-RHF(mol, auxbasis)
2: $(c, E_{\mathrm{CASCI}}) \leftarrow$ DFCASCI($\phi, n_a, n_{\mathrm{el}}$) ▷ sharing ϕ's fitted integrals
3: $(C, c, \varepsilon) \leftarrow$ CANONICALIZE(cas_natorb) ▷ Eq. (5)
4: refuse a degenerate state (`_check_degeneracy`)
5: write $c, \Gamma^{(1)}, \Gamma^{(2)}, h, h', \varepsilon, B_{aa}, B_{ca}, B_{va}, B_{cv}$, PySCF DF-NEVPT2's $(N_\kappa, E_\kappa, E^{(2)})$

Run time (`nevpt2_df_demo --golden f --tiles T [--batch β]`)

6: load and check the golden; refuse it if any B block is missing
7: *res* $\leftarrow$ `DeviceResources::create`(0) ▷ one stream, pool, BLAS and solver handles
8: $\mathcal{S} \leftarrow$ `DfIntegralSource`$(B_{aa}, B_{ca}, B_{va}, B_{cv})$; $v \leftarrow \mathcal{S}$.ACTIVEH2E()
9: $(\Gamma^{(3)}, F^{\mathrm{ca}}, F^{\mathrm{ac}}) \leftarrow$ Algorithm 2(c, v, T)
10: upload $h, h', \varepsilon, \Gamma^{(1)}, \Gamma^{(2)}$
11: $(N_\kappa, E_\kappa, E^{(2)}) \leftarrow$ Algorithm 3$(\mathcal{S}, \beta)$
12: **PASS** iff, for all κ, $|E_\kappa - E_\kappa^{\mathrm{ref}}| < 10^{-7}$, $|N_\kappa - N_\kappa^{\mathrm{ref}}| < 10^{-7}$, $N_\kappa \geq -10^{-7}$, and $|E^{(2)} - E^{(2)}_{\mathrm{ref}}| < 10^{-7}$ (and every block is right under `--check-blocks`)

ctest entries are `nevpt2_df_cas44`, `_cas88`, `_cas1010`, `_ccpvtz_cas1010`, `_cas1212` (`slow`), `_salicylaldimine_cas88` (gitignored golden, registered only once it has been generated), and the digest variants of `_cas1010`.

9 Path 2: PC-NEVPT2 on a CASCI reference

Offline. The state is Path 1's. In addition, unless the golden is written `--no-pc`, `_check_pc_conditioning` must find every class's metric gapped, and block2's PC-NEVPT2 on the same state, with the fitted integrals injected, is stored as `pc_class_energies` and `e_pc_total`.

Algorithm 6 Path 2, PC on CASCI orbitals

Offline: Algorithm 5's lines 1–5, then

1: refuse an ungapped metric (`_check_pc_conditioning`; otherwise `--no-pc`)
2: write block2's $E_\kappa^{\mathrm{PC,ref}}$ and $E^{\mathrm{PC,ref}}$

Run time (`nevpt2_df_demo --golden f --tiles T --pc [--batch β]`)

3: refuse a golden without `pc_class_energies`/`e_pc_total` ▷ before any device work
4: Algorithm 5's run-time lines up to the uploads, unchanged
5: $(\mathrm{status}_\kappa, E_\kappa^{\mathrm{PC}}) \leftarrow$ Algorithm 4$(\mathcal{S}, \beta)$
6: **PASS** iff every class is DONE, $|E_\kappa^{\mathrm{PC}} - E_\kappa^{\mathrm{PC,ref}}| < 10^{-7}$ for all κ, and $\left|\sum_\kappa E_\kappa^{\mathrm{PC}} - E^{\mathrm{PC,ref}}\right| < 10^{-7}$

The equations are Eqs. (3), (8)–(15), (21) and (29)–(38). The total is checked on its own, because eight per-class errors just under 10^{-7} could add up past it. The PASS line is worded differently from SC's, so that a `*_pc` entry cannot pass on SC output. The ctest entries are `nevpt2_df_cas44_pc`, `_cas88_pc`, `_cas1010_pc`, `_ccpvtz_cas1010_pc`, the `--ozaki` variants `_cas88_pc_ozaki` and `_ccpvtz_cas1010_pc_ozaki`, and `_salicylaldimine_cas88_pc`. There is no DF CAS(12,12) PC entry: that golden is `--no-pc`.

10 Path 3: SC-NEVPT2 on a CASSCF reference

Offline, in two steps. First, `--casscf --mo-out` runs one symmetry-adapted `DFCASSCF` that shares the RHF's fitted integrals, and commits the converged orbitals C^{SCF} and E_{CASSCF} as a `.mogold` sidecar. Second, `--mo-from` runs Path 1's deterministic `DFCASCI` on C^{SCF}, which is already in core/active/virtual order and is not re-sorted. Before anything is written, three refusals run. The sidecar's fingerprint must match the run's flags. The orbitals must be symmetry-pure in the expected irreps. And the CASCI must be a converged CASSCF stationary point:

$$\|g^{\mathrm{orb}}(C^{\mathrm{SCF}})\| \le \texttt{CASSCF_GRAD_TOL}, \qquad \left|E_{\mathrm{CASCI}}(C^{\mathrm{SCF}}) - E_{\mathrm{CASSCF}}\right| \le \texttt{CASSCF_E_TOL}, \tag{39}$$

where g^{orb} is the CASSCF orbital gradient (`_check_casscf_stationary`; the thresholds are constants in `generate_golden.py`, and what they were set against is `docs/reference-data.md`, "The `--casscf` / `--mo-from` gate thresholds"). Everything after the solve is Path 1's: the canonicalisation of Eq. (5) (now on CASSCF orbitals), the degeneracy check, the RDMs, the integral blocks and B (Eq. (3) with $C = C^{\mathrm{SCF}}$ canonicalised), and PySCF's DF-NEVPT2. The header records `reference=casscf`.

Algorithm 7 Path 3, SC on CASSCF orbitals

Offline, step 1 (`--df --casscf --mo-out m`; the only place an orbital optimiser runs)
1: $\phi \leftarrow$ DF-RHF; $(C^{\mathrm{SCF}}, E_{\mathrm{CASSCF}}) \leftarrow \mathrm{DFCASSCF}(\phi, n_a, n_{\mathrm{el}})$
2: write the sidecar m: C^{SCF}, E_{CASSCF}, fingerprint
Offline, step 2 (`--df --mo-from m`, the same case flags)
3: check the fingerprint, the orbital symmetry and the irreps
4: $(c, E_{\mathrm{CASCI}}) \leftarrow \mathrm{DFCASCI}(\phi, n_a, n_{\mathrm{el}};\ C^{\mathrm{SCF}})$
5: refuse unless Eq. (39) holds
6: Algorithm 5's offline lines 3–5, with `reference_kind = casscf`
Run time: Algorithm 5's run-time lines, **unchanged**

The equations are Path 1's together with Eq. (39). Nothing at run time depends on the orbitals having been optimised. In particular the SC formulas take ε and h' as given, so a CASSCF golden goes through the same 10^{-7} comparison against PySCF's DF-NEVPT2 on the same CASSCF orbitals. The ctest entry is `nevpt2_df_cas44_casscf` (N2/cc-pVDZ CAS(4,4), `small`, with `--check-blocks`). It is the only density-fitted CASSCF golden. The CAS(10,10) CASSCF golden is conventional, so only `nevpt2_demo` runs it.

11 Path 4: PC-NEVPT2 on a CASSCF reference

Offline. Path 3's two steps, plus Path 2's PC conditioning check and block2 run on the resulting state.

Algorithm 8 Path 4, PC on CASSCF orbitals

Offline: Algorithm 7's two steps, then Algorithm 6's offline lines 1–2 on that state
Run time: Algorithm 6's run-time lines, **unchanged**

The equations are Eqs. (29)–(38) on top of Path 3's. The rank cut, the gap test (Eq. (35)) and the denominator test all run as in Path 2. Whether a CASSCF state's metric is gapped is decided by the same offline check that every other PC golden passes. The ctest entry is `nevpt2_df_cas44_casscf_pc`.

12 Scope and limits

- **The one-electron side is not fitted at run time.** h, h', ε, $\Gamma^{(1)}$ and $\Gamma^{(2)}$ come from the golden. The demo rebuilds only the two-electron integrals from B: the active block v for the RDM build and the intermediates, and every external slab for the classes.
- **A different reference from the conventional goldens.** The `*_df` goldens hold a different (fitted) CASCI state and PySCF's DF-NEVPT2. They are never compared with the conventional numbers, which differ by the fitting error (`docs/reference-data.md`, "Density fitting").
- **CASSCF coverage is one size.** The DF CASSCF paths are tested on N2/cc-pVDZ CAS(4,4) only.
- **PC can refuse.** A class with an ungapped metric, or with a non-positive denominator, gets no energy. A golden whose metric is ungapped carries no PC fields, and `--pc` refuses it before the RDM build.
- **Beyond the energy.** The DF demo also runs `--sc-tangent` and `--gradient-source` (SC only, orbital energies frozen). Both accept a `casscf` golden, since they differentiate at fixed orbitals. PC's CI source term and the CI-block Hessians have no DF slab path. See `docs/gradients.md`.

A The RDM-only intermediates

These are the metric and Dyall blocks of Tables 3 and 4, generated from the einsum chains in `src/energy/energy.cpp` (shared by SC and PC) and `src/energy/energy_sc_classes.cppm` (SC only). Each subscript string was rendered mechanically, sums written out, so that this appendix is the code and not a re-derivation. v is the physicist active block (Eq. (1)), h the active `h1eff`, and δ the identity. Each block is built once per class, before the slab walk, and freed when the class returns.

A.1 Hole-side metrics and the $\bar{\Gamma}$, R families

$$\Gamma^{\mathrm{h}(3)}_{abcdef} = 2\,\Gamma^{(2)}_{abef}\,\delta_{cd} - \Gamma^{(3)}_{abdcef}, \tag{40}$$

$$\Gamma^{\mathrm{h}(2)}_{abcd} = 2\,\Gamma^{(1)}_{ab}\,\delta_{cd} - \Gamma^{(2)}_{abdc}, \tag{41}$$

$$\bar{\Gamma}^{(1)}_{ij} = 2\,\delta_{ij} - \Gamma^{(1)}_{ji}, \tag{42}$$

$$\Gamma'_{ijkl} = \Gamma^{(2)}_{ikjl} - \delta_{jk}\,\Gamma^{(1)}_{il}, \tag{43}$$

$$\bar{\Gamma}^{(2)}_{ijkl} = \Gamma'_{klij} + \delta_{il}\Gamma^{(1)}_{kj} + \delta_{jk}\Gamma^{(1)}_{li} - 2\,\delta_{ik}\Gamma^{(1)}_{lj} - 2\,\delta_{jl}\Gamma^{(1)}_{ki} - 2\,\delta_{il}\delta_{jk} + 4\,\delta_{ik}\delta_{jl}, \tag{44}$$

$$\begin{aligned}\bar{\Gamma}^{(3)}_{pqrabc} = {} & -\delta_{pb}\bar{\Gamma}^{(2)}_{qrac} - \delta_{br}\bar{\Gamma}^{(2)}_{pqac} + 2\,\delta_{bq}\bar{\Gamma}^{(2)}_{prac} + 2\,\delta_{ap}\Gamma^{(2)}_{bqcr} - 4\,\delta_{ap}\delta_{cr}\Gamma^{(1)}_{bq} \\ & + 2\,\delta_{cr}\Gamma^{(2)}_{bqap} - \Gamma^{(3)}_{bqapcr} + 2\,\delta_{ar}\delta_{pc}\Gamma^{(1)}_{bq} - \delta_{ar}\Gamma^{(2)}_{bqcp},\end{aligned} \tag{45}$$

$$R^{(2)}_{ijkl} = \Gamma^{(2)}_{iljk} - \Gamma^{(1)}_{ik}\,\delta_{jl}, \tag{46}$$

$$R^{(3)}_{ijklmn} = \Gamma^{(3)}_{injmkl} - \delta_{jn}\Gamma^{(2)}_{imkl} - \delta_{km}R^{(2)}_{ijln} - \delta_{kn}R^{(2)}_{ijml}. \tag{47}$$

$\Gamma^{\mathrm{h}(1)} = 2\delta - \Gamma^{(1\,\mathsf{T})}$, the same expression as $\bar{\Gamma}^{(1)}$, is built inline in `energy_Si`.

A.2 A^{16} (Sr), reading F^{ca} and F^{ac}

$$\begin{aligned}
A^{16}_{pqrabc} = &-\sum_i h_{ib}\Gamma^{(3)}_{rpqiac} + \sum_i h_{ia}\Gamma^{(3)}_{rpqbic} - \sum_i h_{ci}\Gamma^{(3)}_{rpqbai} - F^{\mathrm{ca}}_{rpacqb} \\
&- \sum_{k,i} v_{kbia}\Gamma^{(3)}_{rpqcki} - \sum_{k,j} v_{kbaj}\Gamma^{(3)}_{rpqjkc} + \sum_{i,j} v_{cbij}\Gamma^{(3)}_{rpqjai} + \delta_{qc}\, f_{prab} \\
&+ F^{\mathrm{ac}}_{rpqbac} - F^{\mathrm{ca}}_{rpqbac} + \sum_{j,i} v_{jbij}\Gamma^{(3)}_{rpqiac} - \sum_{j,k} v_{cjka}\Gamma^{(3)}_{rpqbjk} + \sum_{j,i} v_{jcij}\Gamma^{(3)}_{rpqbai}, \qquad (48)
\end{aligned}$$

$$f_{prab} = \sum_{k,i,j} v_{kbij}\,\Gamma^{(3)}_{rpajki}. \qquad (49)$$

The $\delta_{qc} f_{prab}$ term is `launchDiagA16`, which implements `a16[:, i, ..., i] += fdm2`. Here the labels p, q, r are the rows of A^{16}, all active. They are not the class's virtual.

A.3 A^{22} (Si), reading F^{ca} and F^{ac}

$$\begin{aligned}
A^{22}_{ijkabc} = &-\sum_p h_{pb}\Gamma^{(3)}_{kipjac} - \sum_p h_{pa}\Gamma^{(3)}_{kibjpc} + \sum_p h_{cp}\Gamma^{(3)}_{kibjap} + \sum_{q,r} v_{cqra}\Gamma^{(3)}_{kibjqr} - \sum_{q,p} v_{qcpq}\Gamma^{(3)}_{kibjap} \\
&- F^{\mathrm{ac}}_{kiacbj} - \delta_{ja}\, f^{a}_{ikbc} - \sum_{p,q} v_{pqab}\Gamma^{(3)}_{kiqjpc} + \sum_{p,r} v_{pcrb}\Gamma^{(3)}_{kiajpr} + \sum_{q,r} v_{cqrb}\Gamma^{(3)}_{kiqjar} \\
&- F^{\mathrm{ac}}_{kibjac} + F^{\mathrm{ca}}_{kibjac} + 2\, h_{jb}\Gamma^{(2)}_{kiac} + 2\sum_{p,r} v_{pjrb}\Gamma^{(3)}_{kiprac} + 2\,\delta_{jb}\, f^{b}_{ikac}, \qquad (50)
\end{aligned}$$

$$f^{a}_{ikbc} = \sum_{p,q,r} v_{pqrb}\,\Gamma^{(3)}_{kiqcpr}, \qquad (51)$$

$$\begin{aligned}
f^{b}_{ikac} = &\sum_p h_{pa}\Gamma^{(2)}_{kipc} - \sum_p h_{cp}\Gamma^{(2)}_{kiap} - \sum_{q,r} v_{cqra}\Gamma^{(2)}_{kiqr} + \sum_{q,p} v_{qcpq}\Gamma^{(2)}_{kiap} \\
&+ \sum_{p,q,r} v_{pqra}\Gamma^{(3)}_{kiqcpr} - \sum_{r,p,q} v_{rcpq}\Gamma^{(3)}_{kiaqrp}. \qquad (52)
\end{aligned}$$

The two δ terms are `launchDiagA22a` (`a22[:, i, :, i, :, :]  -= fdm2a`) and `launchDiagA22b` (`a22[:, i, :, :, i, :]  += 2 fdm2b`). All of i, j, k here are active labels of A^{22}.

A.4 The small blocks: A^{3} (Sijr), K^{27} (Srsi)

$$A^{3}_{pa} = \sum_i h_{ia}\bar{\Gamma}^{(1)}_{ip} + 2\sum_{i,j,k} v_{ijka}\delta_{pj}\Gamma^{(1)}_{ik} - \sum_{i,j,k} v_{ijka}\Gamma^{(2)}_{jpik}, \qquad (53)$$

$$K^{27}_{pa} = -\sum_i h_{ai}\Gamma^{(1)}_{pi} - \sum_{i,j,k} v_{iajk}\Gamma^{(2)}_{pkij} + \sum_{i,j} v_{iaji}\Gamma^{(1)}_{pj}. \qquad (54)$$

A.5 A^{7} (Srs), A^{9} (Sij)

$$A^{7}_{pqab} = -\sum_i h_{bi}R^{(2)}_{pqia} - \sum_i h_{ai}R^{(2)}_{pqbi} - \sum_{k,i,j} v_{kbij}R^{(3)}_{pqkija} - \sum_{k,i,j} v_{kaij}R^{(3)}_{pqkibj} - \sum_{i,j} v_{baij}R^{(2)}_{pqij}, \qquad (55)$$

$$\begin{aligned}
A^{9}_{pqab} = &\sum_i h_{ib}\bar{\Gamma}^{(2)}_{pqai} + 2\sum_{i,j} v_{ijib}\bar{\Gamma}^{(2)}_{pqaj} - \sum_{i,j} v_{ijjb}\bar{\Gamma}^{(2)}_{pqai} - \sum_{i,j,k} v_{ijkb}\bar{\Gamma}^{(3)}_{pkqaij} \\
&+ \sum_i h_{ia}\bar{\Gamma}^{(2)}_{pqib} - \sum_{i,j} v_{ijja}\bar{\Gamma}^{(2)}_{pqib} - \sum_{i,j} v_{ijba}\bar{\Gamma}^{(2)}_{pqji} + 2\sum_{i,j} v_{ijia}\bar{\Gamma}^{(2)}_{pqjb} - \sum_{i,j,k} v_{ijka}\bar{\Gamma}^{(3)}_{pqkjbi}. \qquad (56)
\end{aligned}$$

A.6 A^{12}, A^{13} (Sir)

$$
\begin{aligned}
A^{12}_{pqab} = &\sum_i h_{ia}\Gamma^{(2)}_{qpib} - \sum_i h_{bi}\Gamma^{(2)}_{qpai} + \sum_{i,j,k} v_{ijka}\Gamma^{(3)}_{qpjbik} - \sum_{k,i,j} v_{kbij}\Gamma^{(3)}_{qpajki} \\
&- \sum_{j,k} v_{bjka}\Gamma^{(2)}_{qpjk} + \sum_{j,i} v_{jbij}\Gamma^{(2)}_{qpai}, \qquad (57)\\
A^{13}_{pqab} = &-\sum_i h_{ia}\Gamma^{(2)}_{qbip} + 2\,h_{pa}\Gamma^{(1)}_{qb} + \sum_i h_{bi}\Gamma^{(2)}_{qiap} - 2\sum_i \delta_{pa}h_{bi}\Gamma^{(1)}_{qi} \\
&- \sum_{i,j,k} v_{ijka}\Gamma^{(3)}_{qbjpik} + \sum_{k,i,j} v_{kbij}\Gamma^{(3)}_{qjapki} + \sum_{l,m} v_{blma}\Gamma^{(2)}_{qmlp} + 2\sum_{k,m} v_{kpma}\Gamma^{(2)}_{qbkm} \\
&- 2\sum_m v_{bpma}\Gamma^{(1)}_{qm} - \sum_{l,k} v_{lbkl}\Gamma^{(2)}_{qkap} - 2\sum_{m,k,l} \delta_{ap}v_{mbkl}\Gamma^{(2)}_{qlmk} + 2\sum_{l,k} \delta_{ap}v_{lbkl}\Gamma^{(1)}_{qk}. \qquad (58)
\end{aligned}
$$

A.7 SC only: the one-body Dyall blocks A^{17}, A^{19} (Sr), A^{23}, A^{25} (Si)

With $\tilde{h}_{mn} = h_{mn} - \sum_j v_{mjjn}$,

$$
\begin{aligned}
A^{17}_{abcp} &= -\sum_i \tilde{h}_{pi}\Gamma^{(2)}_{cabi} - \sum_{k,i,j} v_{kpij}\Gamma^{(3)}_{cabjki}, \qquad (59)\\
A^{19}_{ap} &= -\sum_i \tilde{h}_{pi}\Gamma^{(1)}_{ai} - \sum_{k,i,j} v_{kpij}\Gamma^{(2)}_{ajki}, \qquad (60)\\
A^{23}_{abcp} &= -\sum_i h_{ip}\Gamma^{(2)}_{caib} - \sum_{i,j,k} v_{pijk}\Gamma^{(3)}_{cajbik} + 2\,h_{bp}\Gamma^{(1)}_{ca} + 2\sum_{i,k} v_{pibk}\Gamma^{(2)}_{caik}, \qquad (61)\\
A^{25}_{ap} &= -\sum_i h_{pi}\Gamma^{(1)}_{ai} - \sum_{i,j,k} v_{pijk}\Gamma^{(2)}_{jaik} + 2\,h_{pa} + 2\sum_{i,j} v_{piaj}\Gamma^{(1)}_{ij}. \qquad (62)
\end{aligned}
$$

PC does not build these four (§7).